



# Backend Senior Technical Challenge

Fintech

Backend Senior

Node.js + TS

48 horas

# Lo que medimos

|                                                |     |
|------------------------------------------------|-----|
| Arquitectura backend y diseño modular          | 25% |
| Transaccionalidad, idempotencia y consistencia | 20% |
| Calidad de código                              | 15% |
| Testing unitario e integración                 | 15% |
| Seguridad y protección de datos                | 10% |
| Documentación técnica                          | 10% |
| Uso responsable de IA                          | 5%  |

# Cómo funciona

## 48 horas

Take Home Challenge

## GitHub / GitLab

Entrega por repositorio con commits claros

## IA Permitida

ChatGPT · Claude · Cursor · Copilot

## Stack obligatorio

Node.js 20+, TypeScript, PostgreSQL, Docker Compose

## Framework sugerido

NestJS preferentemente; se acepta Express/Fastify bien estructurado

## Entorno local

Debe ejecutarse con un solo comando documentado

# Módulo a construir

## Wallet Transaction Service

Construir un microservicio backend para gestionar operaciones de una billetera digital regulada.

### Consultar saldo

Disponible por wallet y moneda

### Crear débito/crédito

Transacción atómica con rollback ante falla

### Revertir transacción

Control de reversa única e impacto en saldo

### Consultar movimientos

Listado paginado, filtros por tipo y estado

### Transferir entre wallets

Doble asiento: débito origen y crédito destino

### Consultar estado

Status transaccional y referencia externa

# Checklist del entregable

## FUNCIONAL

- Login simulado con JWT
- Wallets y saldos iniciales precargados
- Listado de movimientos paginado
- Débito, crédito y transferencia
- Reversa de transacciones
- Health check y readiness
- Swagger / OpenAPI

## TÉCNICO

- Idempotency-Key obligatorio en operaciones críticas
- Manejo de errores centralizado
- Rollback transaccional
- Pruebas unitarias e integración
- Auditoría técnica mínima
- Docker Compose funcional
- Declaración de uso de IA

# Contrato y estructura técnica

## API contract mínimo

### AUTH

```
// POST /auth/login
{
  "username": "senior.backend",
  "password": "Password123"
}

// Response
{ "token": "mock-jwt-token", "expiresIn": 3600 }
```

### BALANCE

```
// GET /wallets/{walletId}/balance
Authorization: Bearer {token}

{
  "walletId": "wal_001",
  "currency": "PEN",
  "availableBalance": "1500.00"
}
```

### MOVEMENTS

```
// GET /wallets/{walletId}/movements
?type=ALL&status=COMPLETED&page=1&pageSize=20

{
  "walletId": "wal_001",
  "total": 2,
  "movements": [{
    "transactionId": "txn_001",
    "amount": "25.50",
    "type": "DEBIT",
    "status": "COMPLETED"
  }]
}
```

# Contrato transaccional

Operaciones críticas con Idempotency-Key

## CREATE TRANSACTION

```
// POST /transactions
Authorization: Bearer {token}
Idempotency-Key: uuid

{
  "walletId": "wal_001",
  "type": "DEBIT",
  "amount": "25.50",
  "currency": "PEN",
  "description": "Pago QR comercio",
  "externalReference": "qr_789456"
}
```

## STATUS

```
// GET /transactions/{transactionId}

{
  "transactionId": "txn_001",
  "status": "COMPLETED",
  "externalReference": "qr_789456"
}
```

## TRANSFER

```
// POST /transactions/transfer
Idempotency-Key: uuid

{
  "sourceWalletId": "wal_001",
  "targetWalletId": "wal_002",
  "amount": "100.00",
  "currency": "PEN",
  "description": "Transferencia entre usuarios"
}
```

## REVERSAL

```
// POST /transactions/{id}/reversal
Idempotency-Key: uuid

{
  "reason": "Merchant refund / reversal",
  "externalReference": "rev_123456"
}
```

# Reglas de negocio críticas

## WALLET

- ✓ Solo wallets ACTIVE pueden operar.
- ✓ No se permite saldo negativo.
- ✓ No se permite operar con monedas distintas.
- ✓ Montos como decimal/string; nunca float para dinero.
- ✓ Saldo disponible deben actualizarse dentro de la misma transacción.

## TRANSACCIONES

- ✓ Toda operación crítica debe ser atómica.
- ✓ Si una parte falla, debe ejecutarse rollback.
- ✓ La misma Idempotency-Key debe devolver la misma respuesta.
- ✓ Misma Idempotency-Key con body diferente debe responder conflicto.
- ✓ Una transacción reversada no puede reversarse nuevamente.
- ✓ Toda operación crítica debe dejar auditoría.

# Calidad y protección

## TESTING PYRAMID

### Integration Test

Obligatorio: transaction, transfer, reversal, balance

### Unit Test

Obligatorio: reglas de negocio, validaciones, idempotencia

## SEGURIDAD

- 🔒 JWT o mock JWT validado por guard.
- 🔒 DTOs con class-validator, zod o equivalente.
- 🔒 Errores controlados sin exponer stack traces.
- 🔒 Logs sin tokens, passwords ni datos sensibles.
- 🔒 Variables de entorno; no secretos reales hardcodeados.
- 🔒 HTTP status codes correctos: 400, 401, 403, 404, 409, 422, 500.
- 🔒 Health check y readiness.

# Qué debes entregar

## ENTREGABLES

- ✓ Repositorio GitHub / GitLab.
- ✓ README con instrucciones claras.
- ✓ Docker Compose para app + PostgreSQL.
- ✓ Migraciones o script de inicialización.
- ✓ Swagger / OpenAPI.
- ✓ Tests unitarios e integración.
- ✓ Diagrama simple de arquitectura.
- ✓ Colección Postman.

## DECLARACIÓN DE IA

## Uso de IA

- Herramienta utilizada:
- ¿Para qué la usaste?
- ¿Qué código aceptaste?
- ¿Qué código descartaste?
- ¿Qué validaste manualmente?
- ¿Qué riesgos identificaste?

El uso de IA no resta puntos. Lo que se evalúa es criterio, revisión, validación y capacidad de explicar decisiones.

# Cómo te evaluamos

## PUNTUACIÓN

|                                  |            |
|----------------------------------|------------|
| Arquitectura backend             | 25         |
| Transaccionalidad e idempotencia | 20         |
| Calidad de código TypeScript     | 15         |
| Testing                          | 15         |
| Seguridad                        | 10         |
| Documentación                    | 10         |
| Uso responsable de IA            | 5          |
| <b>Total</b>                     | <b>100</b> |

## ESCALA

- 90 - 100 **Senior A**
- 85 - 89 **Senior B+**
- 80 - 84 **Senior B**
- 75 - 79 **Semi Senior Alto**
- 65 - 74 **Semi Senior**
- < 65 **No recomendado**

# Casos mínimos que debe cubrir

## HAPPY PATH

- ✓ Débito exitoso.
- ✓ Crédito exitoso.
- ✓ Transferencia exitosa.
- ✓ Reversa exitosa.
- ✓ Consulta de saldo posterior a cada operación.
- ✓ Reintento con misma Idempotency-Key devuelve misma respuesta.

## CASOS BORDE

- ✓ Débito con saldo insuficiente.
- ✓ Wallet bloqueada.
- ✓ Wallet inexistente.
- ✓ Moneda distinta entre wallets.
- ✓ Intento de revertir una transacción ya reversada.
- ✓ Misma Idempotency-Key con body diferente responde 409 Conflict.
- ✓ Error controlado sin exponer información sensible.